

Harti CFR

Stațiile și trecerile la nivel CFR pe o singură hartă

Documentația proiectului și ghid de utilizare

Cosmin Mocanu

19 octombrie 2026

Cuprins

1 Ce este Harti CFR? 2
1.1 Proiectul pe scurt . . . 2

2 Cum se leagă componentele 2

3 Structura proiectului 3

4 Rularea proiectului pe calculatorul tău 3
4.1 Ce trebuie instalat . . . 3
4.2 Pasul 1 – Descarcă codul . . . 3
4.3 Pasul 2 – Pornește baza de date . . . 4
4.4 Pasul 3 – Pornește backend-ul . . . 4
4.5 Pasul 4 – Configurează cheia pentru hartă . . . 4
4.6 Pasul 5 – Pornește frontend-ul . . . 4
4.7 Pasul 6 – Încarcă datele . . . 5
4.8 Rezumat rapid . . . 5

5 Utilizarea aplicației 5
5.1 Crearea unui cont și autentificarea . . . 5
5.2 Dashboard-ul . . . 6
5.3 Filtrarea tabelelor . . . 6
5.4 Selectarea rândurilor și afișarea lor pe hartă . . . 6
5.5 Citirea hărții . . . 7

6 Datele 7
6.1 Treceri la nivel (tabelul `Logs`) . . . 7
6.2 Stații (tabelul `Stations`) . . . 8
6.3 Utilizatori (tabelul `Users`) . . . 8

7 Referința API a backend-ului 8

8 Cum funcționează codul 9
8.1 Backend . . . 9
8.2 Frontend . . . 10

9 Referința de configurare 11
9.1 Frontend (`HartiCFR_FE/.env.local`) . . . 11
9.2 Backend . . . 11

10 Rezolvarea problemelor 11

11 Limitări cunoscute și idei 12

1 Ce este Harti CFR?

Harti CFR este o aplicație web pentru consultarea a două seturi de date ale rețelei feroviare din România (CFR) și afișarea lor pe o hartă interactivă:

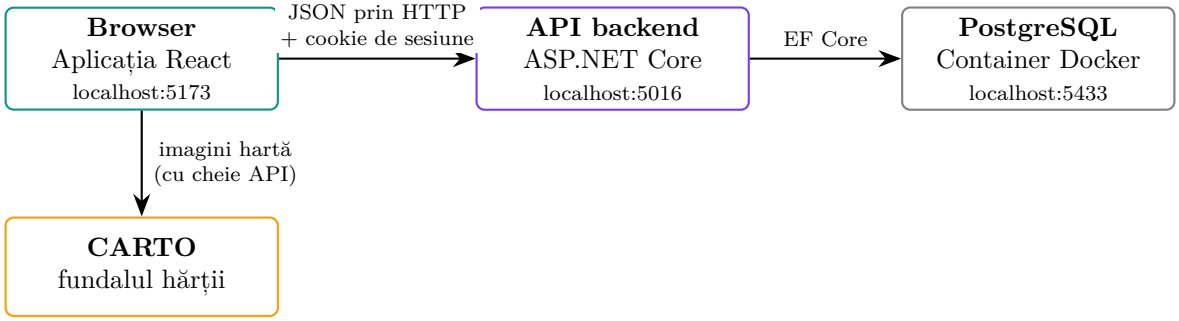
- **Treceri la nivel**: aproximativ 1 200 de treceri la nivel și instalații din stații, dotate cu echipamente de semnalizare. Pentru fiecare, aplicația reține sucursala regională (SRCF), secția CT, poziția kilometrică, stațiile între care se află, magistrala, tipul instalației de semnalizare, producătorul, regimul de funcționare și coordonatele GPS.
- **Stații**: aproximativ 2 300 de stații de cale ferată, fiecare cu codul, numele și coordonatele GPS.

Utilizatorul își creează un cont, se autentifică, filtrează tabelele până rămân înregistrările care îl interesează, le selectează și apasă **Render** pentru a le vedea ca puncte pe harta României. Ținând mouse-ul deasupra unui punct, se afișează toate detaliile înregistrării respective.

1.1 Proiectul pe scurt

Componentă	Tehnologie
Frontend	React 19, TypeScript, Vite, React Router, Leaflet (hartă), SCSS / Tailwind
Backend	ASP.NET Core (.NET 10) Web API, Entity Framework Core
Bază de date	PostgreSQL 16 (rulează local în Docker)
Fundal hartă	Harta de bază CARTO „Voyager” (necesită o cheie API gratuită)
Autentificare	Sesiuni pe bază de cookie, parole criptate cu BCrypt

2 Cum se leagă componentele



1. **Frontend-ul** este o aplicație React de tip single-page. Nu comunică niciodată direct cu baza de date; apelează backend-ul la adresele `/api/...`.
2. **Backend-ul** verifică cine este autentificat (prin cookie-ul `harticfr_auth`), citește datele din PostgreSQL și le returnează în format JSON.
3. **Baza de date** conține trei tabele: `Users` (utilizatori), `Logs` (trecerile la nivel) și `Stations` (stațiile). Backend-ul creează și actualizează automat aceste tabele la fiecare pornire.
4. **Fundalul hărții** este descărcat de browser direct de la CARTO. Doar punctele de pe hartă provin din datele noastre.

3 Structura proiectului

```
HartiCFR/
|-- HartiCFR_BE/                  Backend (ASP.NET Core Web API)
|   |-- Program.cs                Pornirea aplicației (DB, autentificare, CORS)
|   |-- Controllers/              Endpoint-urile HTTP (Auth, Log, Station)
|   |-- Repositories/             Accesul la baza de date, o clasă pe tabel
|   |-- Models/                   Clasele de date (User, Log, Station, enumerări)
|   |-- Data/AppDbContext.cs      Maparea tabelelor în Entity Framework
|   |-- Migrations/               Istoricul schemei bazei de date (generat automat)
|   |-- Configuration/            Transformă un URL postgres:// în connection string
|   |-- appsettings*.json         Configurare (connection string, CORS)
|   `-- Dockerfile                Imaginea de container folosită în producție
`-- HartiCFR_FE/                  Frontend (React + Vite)
    |-- src/
    |   |-- App.tsx               Rutele paginilor
    |   |-- pages/                Pagina de start, Login, Creare cont, Dashboard
    |   |-- components/           DataPanel (tabele), GeoMap (hartă), ...
    |   |-- lib/api.ts            Adresa backend-ului pentru cereri
    |   `-- types/                Structurile TypeScript pentru Log și Station
    |-- public/                   Logo și iconițe
    |-- .env.example              Descrie variabilele de mediu
    `-- package.json              Dependențe și scripturi
```

4 Rularea proiectului pe calculatorul tău

4.1 Ce trebuie instalat

Unealtă	Versiune	Folosită pentru
.NET SDK	10.x	Compilarea și rularea backend-ului
Node.js	20 sau mai nou	Rularea frontend-ului
pnpm	11.x	Instalarea pachetelor frontend (`npm i -g pnpm`)
Docker Desktop	o versiune recentă	Rularea PostgreSQL
Git	orice	Descărcarea codului

Verifică dacă totul este disponibil:

```
dotnet --version
node --version
pnpm --version
docker --version
```

4.2 Pasul 1 – Descarcă codul

```
git clone https://github.com/Cosmin270/HartiCFR.git
cd HartiCFR
```

4.3 Pasul 2 – Pornește baza de date

Backend-ul se așteaptă ca PostgreSQL să ruleze pe portul **5433** (nu pe portul implicit 5432, ca să nu intre în conflict cu un Postgres pe care l-ai avea deja instalat), cu utilizatorul `postgres` și parola `cfr`. Asigură-te că Docker Desktop este pornit.

Doar prima dată – creează containerul:

```
docker run --name harticfr-db \
  -e POSTGRES_PASSWORD=cfr \
  -p 5433:5432 \
  -d postgres:16
```

De fiecare dată după aceea – doar repornește-l:

```
docker start harticfr-db
```

Nu trebuie să creezi tu baza de date `harticfr` sau vreun tabel: backend-ul face asta la pornire (vezi secțiunea 8.1.1).

4.4 Pasul 3 – Pornește backend-ul

```
cd HartiCFR_BE
dotnet run --launch-profile http
```

Așteaptă până apare mesajul `Now listening on: http://localhost:5016`. Verifică dacă funcționează deschizând `http://localhost:5016/health` – răspunsul trebuie să fie `{"status":"ok"}`.

4.5 Pasul 4 – Configurează cheia pentru hartă

Fundalul hărții vine de la CARTO, care cere o cheie API gratuită. Fără ea, fiecare porțiune de hartă afișează un filigran mare „API KEY REQUIRED”.

1. Cere o cheie la `https://carto.com/basemaps/apikey`.
2. Creează fișierul `HartiCFR_FE/.env.local` cu conținutul:

   ```
   VITE_CARTO_API_KEY=cheia-ta-aici
   ```

Fișierul `.env.local` este ignorat de Git, deci cheia nu ajunge niciodată într-un commit.

4.6 Pasul 5 – Pornește frontend-ul

Deschide un al doilea terminal (backend-ul continuă să ruleze în primul):

```
cd HartiCFR_FE
pnpm install      # prima dată sau după modificări în package.json
pnpm dev
```

Deschide `http://localhost:5173` în browser.

Sfat

Folosește `localhost`, nu `127.0.0.1`. Serverul de dezvoltare ascultă doar pe `localhost`, iar cookie-ul de autentificare este legat de acest nume.

4.7 Pasul 6 – Încarcă datele

O bază de date nou creată are tabelele, dar **nu are nicio stație și nicio trecere la nivel**, așa că tabelele din dashboard vor fi goale. Proiectul nu conține un script de import pentru fișierele sursă (`Statii Romania_practica.csv` și `Treceri cu instalatii de semnalizare_PRACTICA.csv`); datele au fost încărcate manual în baza de date locală a dezvoltatorului.

Cel mai simplu mod de a obține datele este să ceri o copie (dump) a unei baze de date existente și să o restaurezi în containerul tău:

```
# pe un calculator care are deja datele
docker exec harticfr-db pg_dump -U postgres -d harticfr \
  --data-only -t '"Stations"' -t '"Logs"' > harticfr-data.sql

# pe calculatorul tău
docker exec -i harticfr-db psql -U postgres -d harticfr < harticfr-data.sql
```

Corespondența dintre coloanele din fișierele CSV și baza de date este descrisă în secțiunea 6.

4.8 Rezumat rapid

Ce	Adresă
Site (frontend)	`http://localhost:5173`
API backend	`http://localhost:5016`
Verificare stare	`http://localhost:5016/health`
PostgreSQL	`localhost:5433`, baza `harticfr`, utilizator `postgres`, parolă `cfr`

5 Utilizarea aplicației

Interfața aplicației este în limba engleză, așa că numele butoanelor sunt scrise mai jos exact cum apar pe ecran.

5.1 Crearea unui cont și autentificarea

1. Deschide site-ul. Pagina de start oferă butoanele **Log in** (autentificare) și **Create an account** (creare cont).
2. Alege **Create an account**, completează numele, adresa de e-mail și parola, apoi apasă **Create account**. Fiecare adresă de e-mail poate fi înregistrată o singură dată.
3. Autentifică-te cu aceeași adresă de e-mail și parolă. Vei ajunge pe **dashboard**.

Atenție

Crearea contului *nu* te autentifică automat. După ce apeși **Create account**, aplicația încearcă pentru scurt timp să deschidă dashboard-ul și apoi te trimite la pagina de autentificare – acest comportament este normal. Autentifică-te cu contul pe care tocmai l-ai creat.

Rămâi autentificat timp de 7 zile; fiecare vizită prelungește sesiunea. Butonul **Log out** din colțul din dreapta sus încheie sesiunea. Dacă deschizi `/dashboard` fără să fii autentificat, ești trimis la pagina de autentificare.

5.2 Dashboard-ul

Dashboard-ul are trei zone:

- **Bara de sus**, cu două tab-uri – **Treceri la nivel** și **Statii** – și butonul **Log out**.
- **Tabelul** pentru tab-ul selectat, cu un filtru sub antetul fiecărei coloane.
- **Harta**, în dreapta pe ecranele mari și sub tabel pe telefoane și tablete.

Deasupra tabelului, trei contoare arată câte înregistrări are tab-ul curent (**Records**), câte rânduri sunt selectate în total, din ambele tab-uri (**Selected**), și câte sunt afișate în acest moment pe hartă (**Plotted**).

5.3 Filtrarea tabelelor

Fiecare coloană are un filtru în al doilea rând al antetului:

- **Filtrele text** (*Filter…*) păstrează rândurile a căror valoare *conține* textul introdus, fără să conteze literele mari sau mici. Dacă scrii `CT1` în *Sectia CT*, rămân toate secțiile care conțin „CT1”.
- **Filtrele cu listă** (*Tip, Instalatie, Config, Producator, Regim*) păstrează doar rândurile cu exact valoarea aleasă. Opțiunea **All** dezactivează filtrul.

Filtrele se combină: un rând este afișat doar dacă îndeplinește *toate* filtrele active. Cât timp există filtre active, titlul tabelului arată „*N of M rows*” (N din M rânduri), iar butonul **Clear filters** le resetează pe toate.

5.4 Selectarea rândurilor și afișarea lor pe hartă

1. Apasă pe un rând (sau pe căsuța lui) pentru a-l selecta. Apasă din nou pentru a-l deselecta.
2. Căsuța din rândul de antet selectează sau deselectează *toate rândurile care corespund filtrelor curente* – o metodă rapidă de a afișa, de exemplu, toate trecerile realizate de un anumit producător.
3. Selecțiile se păstrează când schimbi tab-ul, deci poți selecta câteva treceri, trece la **Statii**, selecta câteva stații și le poți afișa împreună.
4. Apasă **Render (N)**. Toate trecerile și stațiile selectate apar pe hartă. Pe telefon, pagina coboară automat până la hartă.

Clear selection debifează toate rândurile din tab-ul curent. Nu șterge punctele deja afișate pe hartă – harta se schimbă doar când apeși din nou **Render**.

5.5 Citirea hărții

- Punctele **verzi-albăstrui** sunt treceri la nivel, iar cele **violet** sunt stații.
- Punctele apropiate sunt grupate într-un cerc cu un număr. Apropie harta (zoom) sau apasă pe cerc ca să-l desfaci; de la nivelul de zoom 13, fiecare punct apare separat.
- **Ține mouse-ul deasupra unui punct** ca să vezi toate detaliile: pentru o trecere – SRCF, secția CT, kilometrul, tipul, magistrala, instalația, configurația, producătorul, regimul de funcționare și coordonatele; pentru o stație – codul și coordonatele.
- După **Render**, harta se încadrează automat astfel încât să cuprindă toate punctele afișate.
- Dacă selectezi un rând care este deja pe hartă, harta „zboară" până la el și îl evidențiază cu **portocaliu**. Rândul corespunzător din tabel este și el evidențiat.
- Etichetele **Treceri** și **Statii** de deasupra hărții arată câte puncte de fiecare tip sunt afișate. Apasă pe o etichetă ca să ascunzi sau să reafișezi stratul respectiv; apasă pe × ca să-l elimini de pe hartă.

Înregistrările fără coordonate valide (lipsă sau 0,0) nu apar pe hartă, fără niciun mesaj; ele rămân însă vizibile în tabele.

6 Datele

6.1 Treceri la nivel (tabelul `Logs`)

Coloană pe ecran	Câmp	Semnificație
ID	`Id`	Număr intern, atribuit de baza de date.
SRCF	`SRCF`	Sucursala regională de căi ferate (de ex. București).
Sectia CT	`CT`	Secția CT responsabilă de trecere.
Km TN	`TNPosition`	Poziția kilometrică pe linie.
Tip	`LocationType`	`LC` (trecere în linie curentă) sau `Statie` (în interiorul unei stații).
De la	`Location.start`	Prima stație lângă / între care se află trecerea.
Pana la	`Location.end`	A doua stație (gol pentru trecerile din stații).
Magistrala	`MainLine`	Numărul magistralei.
Instalatie	`ElectricType`	Instalația de semnalizare: `BAT` sau `SAT`.
Config	`ConfigType`	`Electronic`, `Relee` sau `Necunoscut`.
Producator	`Producer`	`Aerofina`, `Electromagnetica`, `SPIACT_Cr` sau `_` când nu se cunoaște.
Regim	`OperatingMode`	`Automat` sau `Manual`.
Longitudine	`Position.x`	Longitudinea GPS (est).
Latitudine	`Position.y`	Latitudinea GPS (nord).

Aceste câmpuri provin din fișierul *Treceri cu instalatii de semnalizare_PRACTICA.csv*: *SRCF, Sectia CT, Pozitia km TN, LC/Statie, Amplasare Statia 1 / Statia 2, Numar magistrala, Tip instalatie, Tip electro/relee, Producator, Regim de functionare, X longitudine, Y latitudine.* Coloanele *In dependenta cu semnale* și *Anul modernizarii* din fișier nu sunt salvate.

6.2 Stații (tabelul `Stations`)

Coloană pe ecran	Câmp	Semnificație
ID	`Id`	Număr intern.
Cod	`Cod`	Codul oficial al stației.
Nume statie	`Name`	Numele stației.
Longitudine	`Position.x`	Longitudinea GPS.
Latitudine	`Position.y`	Latitudinea GPS.

Acestea provin din fișierul *Statii Romania_practica.csv* (`cod, nume, xcoord, ycoord`).

6.3 Utilizatori (tabelul `Users`)

Fiecare cont reține un nume de utilizator, o adresă de e-mail unică, un hash BCrypt al parolei (parola propriu-zisă nu este salvată niciodată) și data creării.

7 Referința API a backend-ului

Toate endpoint-urile se află la adresa backend-ului (`http://localhost:5016` local). Corpurile cererilor și răspunsurilor sunt în format JSON. Endpoint-urile marcate în coloana **Auth** returnează `401 Unauthorized` dacă cererea nu conține cookie-ul de sesiune obținut prin `/api/auth/login`.

Metodă	Cale	Auth	Ce face
GET	`/health`	nu	Returnează `{"status":"ok"}`; folosit pentru a verifica dacă serverul funcționează.
POST	`/api/auth/register`	nu	Creează un cont. Corp: `username`, `email`, `password`. `400` dacă e-mailul există deja.
POST	`/api/auth/login`	nu	Corp: `email`, `password`. Setează cookie-ul de sesiune și returnează `id`, `username`, `email`. `401` la date greșite.
GET	`/api/auth/me`	da	Returnează `userId` și `username` pentru utilizatorul autentificat.
POST	`/api/auth/logout`	nu	Șterge cookie-ul de sesiune.
GET	`/api/Log/logs`	da	Listează toate trecerile la nivel.
GET	`/api/Log/logs/{id}`	da	O singură trecere sau `404`.
POST	`/api/Log/logs`	da	Adaugă o trecere (corp: un obiect trecere). Nu este folosit de site.
GET	`/api/Station/stations`	da	Listează toate stațiile.
GET	`/api/Station/stations/{id}`	da	O singură stație sau `404`.

Testarea API-ului din terminal (fișierul `jar.txt` păstrează cookie-ul, deci rămâi autentificat între comenzi):

```
curl -c jar.txt -H 'Content-Type: application/json' \
  -d '{"email":"tu@exemplu.ro","password":"secret"}' \
  http://localhost:5016/api/auth/login

curl -b jar.txt http://localhost:5016/api/Station/stations/1
```

O trecere la nivel este returnată astfel (valorile enumerărilor sunt trimise ca text):

```
{
  "id": 1, "srcf": "Bucuresti", "ct": "CT1 Bucuresti",
  "tnPosition": 5240, "locationType": "Statie",
  "location": { "start": "H. Pajura", "end": "" },
  "mainLine": "", "electricType": "SAT", "configType": "Relee",
  "producer": "_", "operatingMode": "Automat",
  "position": { "x": 26.0361, "y": 44.4728 }
}
```

8 Cum funcționează codul

8.1 Backend

Backend-ul are o structură simplă *controller* → *repository* → *bază de date*.

`Program.cs`
: Leagă totul: citește conexiunea la baza de date, înregistrează repository-urile, configurează autentificarea prin cookie și CORS, aplică migrările și mapează controller-ele și `/health`.

`Controllers/`
: O clasă pentru fiecare resursă. `AuthController` se ocupă de conturi și sesiuni; `LogController` și `StationController` returnează datele și sunt marcate cu `[Authorize]`, deci cer un utilizator autentificat.

`Repositories/`
: Fiecare repository (cu interfața corespunzătoare) conține interogările Entity Framework pentru un tabel. Controller-ele folosesc doar interfețele, astfel încât codul pentru baza de date stă într-un singur loc.

`Models/`
: Clase C# simple pentru tabele. `Helper.cs` conține enumerările (`LocationType`, `ElectricType`, `ConfigType`, `Producer`, `OperatingMode`) și tipurile valoare `Location` și `Position`.

`Data/AppDbContext.cs`
: Îi spune lui Entity Framework cum să salveze modelele: `Location` și `Position` devin coloane separate (`Position_x`, `Position_y`, …), iar enumerările sunt salvate ca text.

Autentificarea. La autentificare, serverul verifică parola față de hash-ul BCrypt și emite un cookie criptat, de tip HTTP-only, numit `harticfr_auth`, valabil 7 zile și prelungit automat la fiecare utilizare. Apelurile API neautentificate primesc `401` (nu o redirecționare), iar frontend-ul folosește acest răspuns pentru a trimite utilizatorul la pagina de autentificare.

CORS. În dezvoltare, orice origine `localhost` poate apela API-ul cu credențiale. În producție, originile permise sunt citite din setarea `Cors__AllowedOrigins`.

8.1.1 Migrările bazei de date

Schema este definită de modelele C# și înregistrată în `Migrations/`. La fiecare pornire, backend-ul rulează `Database.Migrate()`, care creează baza de date dacă este nevoie și aplică migrările care nu au fost încă aplicate. După ce modifici un model, creează o migrare nouă:

```
dotnet tool install --global dotnet-ef     # o singura data
cd HartiCFR_BE
dotnet ef migrations add DescriereaModificarii
```

Migrarea va fi aplicată automat la următoarea pornire a backend-ului.

8.2 Frontend

`src/App.tsx`
: Definește paginile: `/` (pagina de start), `/login`, `/signin` (creare cont) și `/dashboard`. Dashboard-ul este protejat de `ProtectedRoute`.

`components/ProtectedRoute.tsx`
: Apelează `/api/auth/me` înainte de a afișa o pagină protejată și redirecționează la `/login` dacă nu există o sesiune.

`pages/DashboardPage.tsx`
: Încarcă o singură dată trecerile și stațiile, reține selecțiile, înregistrările afișate și punctul evidențiat pe hartă și definește coloanele fiecărui tabel (`LOG_COLUMNS`, `STATION_COLUMNS`).

`components/DataPanel.tsx`
: Un tabel reutilizabil cu filtre pe fiecare coloană, selecție de rânduri și butonul **Render**. Rândurile sunt memoizate, astfel încât bifarea unei căsuțe nu redesenează mii de rânduri.

`components/GeoMap.tsx`
: Harta Leaflet: stratul de fundal, punctele grupate pentru fiecare strat, ferestrele cu detalii, încadrarea automată și deplasarea către punctul selectat.

`lib/api.ts`
: Construiește adresa completă pentru fiecare cerere API (vezi `VITE_API_URL` mai jos). Toate cererile trimit `credentials: "include"`, astfel încât cookie-ul de sesiune să însoțească cererea.

Comenzi utile pentru frontend (rulate în `HartiCFR_FE`):

`pnpm dev`	Server de dezvoltare cu reîncărcare automată, pe portul 5173.
`pnpm build`	Verifică tipurile și construiește site-ul de producție în `dist/`.
`pnpm preview`	Servește local folderul `dist/` construit.
`pnpm lint`	Rulează linter-ul Oxlint.

9 Referința de configurare

9.1 Frontend (`HartiCFR_FE/.env.local`)

Variabilă	Semnificație
`VITE_CARTO_API_KEY`	Cheia CARTO pentru fundalul hărții. Necesară pentru o hartă fără filigran. Ajunge în codul JavaScript construit, deci este publică prin natura ei.
`VITE_API_URL`	Adresa de bază a backend-ului. Lasă-o nesetată local (implicit `http://localhost:5016`). Se setează doar dacă frontend-ul trebuie să folosească un backend de la altă adresă.

Vite citește aceste variabile la pornirea serverului de dezvoltare sau la construirea site-ului, deci repornește `pnpm dev` (sau reconstruiește site-ul) după ce le modifici.

9.2 Backend

Setare	Semnificație
`ConnectionStrings:DefaultConnection`	Conexiunea la baza de date folosită local; setată în `appsettings.Development.json`.
`DATABASE_URL` (variabilă de mediu)	Dacă este setată, o înlocuiește pe cea de mai sus. Acceptă un URL de forma `postgres://user:parola@host:port/db`.
`Cors:AllowedOrigins`	Listă separată prin virgule cu originile suplimentare care pot apela API-ul (ca variabilă de mediu: `Cors__AllowedOrigins`).
`ASPNETCORE_ENVIRONMENT`	`Development` local (reguli mai relaxate pentru cookie și CORS, documentul OpenAPI la `/openapi/v1.json`); `Production` în Docker.
`PORT` (variabilă de mediu)	Portul pe care ascultă containerul Docker (implicit 8080).

10 Rezolvarea problemelor

Problemă	Ce faci
Backend-ul se oprește la pornire cu o eroare de conexiune	Baza de date nu rulează. Pornește Docker Desktop, apoi `docker start harticfr-db`.
`Cannot connect to the Docker daemon`	Docker Desktop nu este deschis. Pornește-l și așteaptă până indică faptul că rulează.
„Could not load data. Is the server running?” pe dashboard	Backend-ul nu rulează sau nu este pe portul 5016. Pornește-l (pasul 3).
Harta afișează „API KEY REQUIRED”	`VITE_CARTO_API_KEY` lipsește sau este greșită în `HartiCFR_FE/.env.local`. Corecteaz-o și repornește `pnpm dev`.

Problemă	Ce faci
Zona hărții este complet gri	Nu există conexiune la internet sau imaginile hărții sunt blocate; verifică uneltele de dezvoltator ale browserului (tab-ul Network).
Tabelele sunt goale („No crossings found”)	Baza de date nu are încă date; vezi secțiunea 4.7.
După crearea contului ajung pe pagina de autentificare	Este normal: înregistrarea nu te autentifică. Autentifică-te cu contul nou.
Autentificarea reușește, dar sunt trimis înapoi la pagina de login	Cookie-ul nu a fost salvat. Local, deschide site-ul pe `localhost`, nu pe `127.0.0.1`.
Portul este deja folosit (5016, 5173 sau 5433)	O altă instanță rulează încă. Oprește-o sau găsește-o cu `lsof -i :5016`.
O înregistrare selectată nu apare pe hartă	Nu are coordonate (sau are 0,0) în baza de date, deci nu poate fi poziționată.

11 Limitări cunoscute și idei

- Înregistrarea nu autentifică automat utilizatorul.
- Nu există resetare a parolei și nici reguli pentru complexitatea parolei sau formatul adresei de e-mail.
- Datele nu pot fi modificate din site. Backend-ul are un endpoint pentru adăugarea unei treceri (`POST /api/Log/logs`), dar nicio pagină nu îl folosește și nu există nimic pentru editare sau ștergere.
- Proiectul nu are un script care să importe fișierele CSV în baza de date; o unealtă de import ar ușura mult configurarea pe un calculator nou.
- Nu există teste automate.